

BIANCA LARA NUNES
LUCAS EDUARDO DE CARVALHO FERREIRA
RUBY NOVAES MARSHALL

SISTEMA DE CATÁLOGO DE PRODUTOS: IMPLEMENTAÇÃO E ANÁLISE DE ALGORITMOS DE BUSCA E ORDENAÇÃO

SÃO SEBASTIÃO DO PARAÍSO – MG
2026

1. Objetivo

Este relatório apresenta os testes e a análise dos algoritmos de busca e ordenação implementados no Sistema de Catálogo de Produtos. O programa armazena produtos em um vetor de estruturas, permite cadastro e consulta e compara o desempenho dos algoritmos por meio de contagens de comparações e movimentações

2. Testes mínimos

Teste	Procedimento	Resultado observado
1- Busca no início	Buscar um produto próximo ao início após ordenar por código.	O produto de código 1007 (Mouse Sem Fio) foi encontrado. Busca sequencial: 1 comparação. Busca binária: 7 comparações.
2- Busca no final	Buscar um produto próximo ao final após ordenar por código.	O produto de código 1097 (Notebook Stand Pro) foi encontrado. Busca sequencial: 14 comparações. Busca binária: 7 comparações.
3- Código inexistente	Buscar um código não cadastrado.	O código 88888 não foi encontrado por nenhuma das buscas. Busca sequencial: 17 comparações. Busca binária: 10 comparações.
4- Códigos únicos e outros valores repetidos	Verificar se o sistema rejeita o cadastro de um produto com código já existente e permite cadastrar outro produto com código único, mesmo que preço, categoria e estoque sejam iguais aos de outro produto.	O cadastro do código 1097 foi rejeitado por duplicidade. O produto de código 1200 (Mouse de Teste) foi aceito com preço R$ 119,50, categoria Periféricos e estoque de 31 unidades.
5- Vetor em ordem crescente	Comparar os quatro algoritmos com os produtos em ordem crescente de código.	Selection Sort: 105 comparações e 0 movimentações; Insertion Sort: 14 comparações e 28 movimentações; Bubble Sort otimizado: 14 comparações e 0 movimentações; Merge Sort: 31 comparações e 118 movimentações.

6- Vetor em ordem decrescente	Comparar os quatro algoritmos com os produtos em ordem decrescente de código.	Selection Sort: 105 comparações e 21 movimentações; Insertion Sort: 105 comparações e 133 movimentações; Bubble Sort otimizado: 105 comparações e 315 movimentações; Merge Sort: 28 comparações e 118 movimentações.

3. Comparação entre busca sequencial e busca binária

A comparação foi realizada após ordenar o vetor por código. Os resultados mostram que a busca binária utilizou menos comparações nos três cenários observados.

Cenário	Código pesquisado	Busca sequencial	Busca binária
Produto próximo ao início	1007	1 comparação — encontrado	7 comparações — encontrado
Produto próximo ao final	1097	14 comparações — encontrado	7 comparações — encontrado
Código inexistente	88888	17 comparações — não encontrado	10 comparações — não encontrado

Observação: os testes foram realizados com 17 produtos cadastrados na sessão, incluindo os produtos de teste 9999 e 99999. Por isso, o código 88888 foi utilizado como código inexistente.

4. Verificação complementar das funcionalidades

- Cadastro com código duplicado: o sistema rejeitou o código 1097 e exibiu a mensagem “ERRO: codigo de produto ja cadastrado”.
- Valores repetidos: o sistema aceitou o produto de código 1200, mesmo com preço, categoria e estoque iguais aos de outro produto.
- Estoque baixo: com limite de 10 unidades, foram listados Webcam Full HD (9), Microfone USB (6), Monitor 24 (8), Fonte 650W (7) e Produto Teste (0).
- Busca binária após ordenação por código: após ordenar os produtos por código com Selection Sort, a busca pelo código 1007 encontrou o produto.
- Busca binária após ordenação por preço: depois de ordenar os produtos por preço crescente, o sistema bloqueou a busca binária e informou que o vetor precisava estar ordenado por código.
- Cadastro após ordenação: ao cadastrar um produto, o sistema informou que a busca binária precisava de nova ordenação por código.

5. Análise dos resultados

1. Qual algoritmo apresenta melhor comportamento quando os dados já estão ordenados?

Nos testes realizados com os dados em ordem crescente, Insertion Sort e Bubble Sort otimizado apresentaram o menor número de comparações, com 14 cada. O Bubble Sort otimizado também realizou zero movimentações, enquanto o Insertion Sort realizou 28. Portanto, ambos tiveram bom desempenho nessa situação, considerando métricas diferentes.

2. Como a ordem inicial dos dados afeta o desempenho?

Insertion Sort e Bubble Sort podem apresentar diferença significativa entre entradas ordenadas e em ordem inversa. Em uma entrada ordenada, ambos podem realizar $n-1$ comparações; em ordem inversa, podem chegar a aproximadamente $n(n-1)/2$ comparações.

3. O Selection Sort é afetado pela ordem inicial dos dados?

O número de comparações do Selection Sort permanece $n(n-1)/2$ para um vetor de n elementos, independentemente da ordem inicial. A quantidade de trocas pode variar conforme a implementação e os dados.

4. Em qual situação a busca sequencial tende a ser mais rápida?

Quando o elemento procurado está próximo do início do vetor, a busca sequencial pode encontrá-lo com poucas comparações. Se estiver no fim ou não existir, pode ser necessário percorrer todo o vetor.

5. Por que a busca binária exige dados ordenados?

A busca binária usa a ordenação para decidir em qual metade continuar a busca. Sem essa propriedade, descartar metade do vetor não é seguro e o algoritmo pode não encontrar um elemento existente.

6. O que acontece com a busca binária quando um novo produto é cadastrado?

Se o produto for inserido ao final do vetor, a ordem crescente por código pode ser quebrada. Por isso, é necessário ordenar novamente antes da busca binária ou manter uma indicação confiável de que o vetor continua ordenado.

7. Quais são as principais diferenças entre os algoritmos de ordenação?

Bubble Sort compara elementos adjacentes e pode parar cedo quando não há trocas. Insertion Sort insere cada elemento na posição correta de uma parte já ordenada. Selection Sort procura o menor elemento restante a cada etapa. Merge Sort divide o vetor em partes, ordena essas partes e depois as intercala; costuma ter complexidade $O(n \log n)$, usando memória auxiliar.

6. Registro das métricas de ordenação

Para concluir a análise experimental, registre para cada algoritmo o número de comparações e movimentações informado pelo programa, usando cópias idênticas do mesmo vetor de entrada. Repita a comparação tanto para os dados em ordem crescente quanto para os dados em ordem decrescente.

Algoritmo	Entrada crescente	Entrada decrescente	Observações
Selection Sort	Comparações: 105 Movimentações: 0	Comparações: 105 Movimentações: 21	Manteve 105 comparações nas duas entradas
Insertion Sort	Comparações: 14 Movimentações: 28	Comparações: 105 Movimentações: 133	Teve menos comparações e movimentações na entrada crescente.
Bubble Sort otimizado	Comparações: 14 Movimentações: 0	Comparações: 105 Movimentações: 315	Foi eficiente na entrada crescente, mas realizou 315 movimentações na decrescente.
Merge Sort	Comparações: 31 Movimentações: 118	Comparações: 28 Movimentações: 118	Realizou 31 comparações na entrada crescente e 28 na decrescente, com 118 movimentações em ambos os cenários.

7. Conclusão

Os resultados mostram que a ordem inicial dos dados influencia o desempenho de alguns algoritmos. Na entrada crescente, Insertion Sort e Bubble Sort otimizado realizaram 14 comparações cada, enquanto na entrada decrescente ambos realizaram 105 comparações. O Selection Sort manteve 105 comparações nos dois cenários. O Merge Sort apresentou 31 comparações na entrada crescente e 28 na decrescente, com 118 movimentações em ambos os casos. O Bubble Sort otimizado registrou 315 movimentações na entrada decrescente, o maior valor observado entre os quatro algoritmos.

Na comparação de buscas, a busca binária utilizou menos comparações nos três cenários observados, mas só pôde ser executada quando o vetor estava ordenado por código. O programa também bloqueou corretamente a busca binária após a ordenação por preço e informou a necessidade de ordenar novamente após cadastrar um produto.

Assim, os testes reforçam a importância de escolher algoritmos adequados e controlar a organização dos dados para manter a confiabilidade e a eficiência do catálogo de produtos.